

20.8.26

CO4 - AT1

NAME: S. Logeshwari

Reg No: 192565040

COMPARATIVE ANALYSIS.

1. Compare the hashing and Binary Search tree based on Time Complexity methods.
2. Compare time complexity of all notations in Bubble, insertion, selection, merge, heap.

1. Answer:

Operation	Hashing Average case	Hashing Worst case	BST Average case	BST Worst case
Search	$O(1)$	$O(n)$	$O(\log n)$	$O(n)$
Insertion	$O(1)$	$O(n)$	$O(\log n)$	$O(n)$
Deletion	$O(1)$	$O(n)$	$O(\log n)$	$O(n)$

Hashing :

* A hash function maps a key directly to a location in a hash table.

* Searching, insertion and deletion are generally $O(1)$ on average.

* In worst case collisions may cause the complexity to become $O(n)$.

Binary Search Tree:

* In operations takes $O(\log n)$ because the search space is reduced at every level.

* However, if the tree become skewed it behaves like a linked list and the complexity becomes $O(n)$.

2.

Sorting Algorithm	Best Case	Average case	Worst Case	Space Complexity
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$

Bubble Sort:

* It repeatedly compares adjacent elements and swaps them if they are in the wrong order.

* It is simple but inefficient for large datasets.

Insertion Sort.

It builds the sorted array one element at a time. It performs well when the data is already sorted.

Selection sort:

- * It repeatedly finds the minimum element from the unsorted portion and places it in the correct position.

- * Its time complexity remains $O(n^2)$ in all cases.

Merge sort

- * It divide and conquer technique.

- * Its time complexity is $O(n \log n)$ in all cases.

Heap sort:

- * It is a binary heap to repeatedly select the largest or smallest element.

- * It provides $O(n \log n)$ in time complexity in best, average and worst cases.

20
21